A Comprehensive Security Framework for Autonomous AI Agents: Integrating Structural Guardrails, Behavioral Zero-Trust, and Post-Quantum Resilience

Yoshihiro Ando
Independent Researcher
yosh5295@gmail.com
Tokyo, Japan

***Abstract*—As Large Language Model (LLM) agents transition from passive chatbots to autonomous entities capable of executing system-level tools via protocols like the Model Context Protocol (MCP), they introduce a critical "Agency Gap." Traditional security perimeters are insufficient for protecting against prompt injection and future cryptographic threats. This paper proposes a unified security framework designed to protect agent-based systems across two core dimensions: Space and Time, while ensuring human-centric governance. First, we establish *Structural Guardrails* (Space) using AST-based static analysis and environment isolation. Second, we implement *Behavioral Zero-Trust* through verifiable workload identity and *White-box Agency*, leveraging Human-in-the-Loop (HITL) not just for security, but for cognitive synchronization and early loop detection. Finally, we ensure *Temporal Sovereignty* (Time) by integrating a dual-layered integrity architecture (Hashed Chain and Merkle Tree) with Post-Quantum Cryptography (PQC). Our evaluation demonstrates that this architecture provides robust, multi-layered defense and enhances agent transparency with minimal computational overhead.**

***Index Terms*—AI Agents, Model Context Protocol (MCP), Agentic Security, Zero Trust, Post-Quantum Cryptography, White-box Agency, Human-in-the-Loop, Merkle Tree.**

I. Introduction

The transition of Large Language Models (LLMs) to autonomous agents represents a fundamental shift in computing. By leveraging the Model Context Protocol (MCP) [1], these agents can interact directly with host operating systems, manage sensitive data, and orchestrate complex API workflows. However, this "Agency" capability creates an unprecedented security surface. Traditional security perimeters are often ineffective against "Prompt Injection" attacks, where an attacker manipulates the LLM's goal-oriented reasoning to execute unauthorized system commands [10].

The OWASP Top 10 for LLM Applications [6] identifies "Insecure Output Handling" (LLM02) and "Excessive Agency" (LLM08) as critical risks. Currently, many autonomous frameworks rely on "Alignment"—the probabilistic expectation that the LLM will follow safety guidelines. We argue that behavioral alignment is insufficient for mission-critical systems. We require a holistic, deterministic security architecture that ensures system integrity and human oversight even if the model's logic is compromised.

This paper presents a "Triple-Lock" framework for Agentic Security, addressing the agent's action through three integrated dimensions:

1) **Structural Containment (Space):** Establishing deterministic safety boundaries using AST inspection and environment isolation.
2) **Behavioral Integrity & White-box Agency (Behavior):** Implementing Zero Trust where every action is verified by cryptographically signed identity and synchronized with human cognition for early detection of "Reasoning Loops."
3) **Temporal Sovereignty (Time):** Protecting against future cryptographic threats using PQC and a dual-layered audit architecture (Hashed Chain for real-time protection and Merkle Tree for session-wide anchoring).

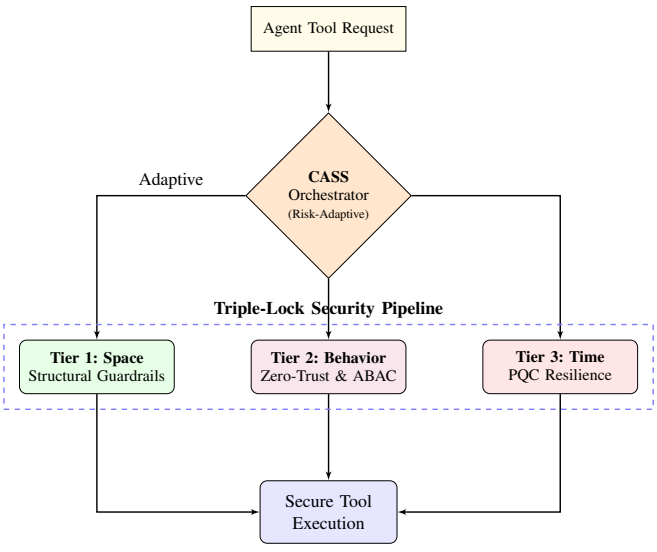

Fig. 1. Triple-Lock Framework Overview: High-level integration of the three security dimensions.

II. Related Work

Existing efforts to secure agents generally fall into three categories: *Behavioral Filtering*, *Structural Isolation*, and *Governance Frameworks*. Behavioral filters like **Guardrails AI** and **LLM-Guard** [2] use secondary LLMs or pattern matching to scrub outputs, but often introduce significant latency and can be bypassed by complex semantic attacks. Isolation patterns, such as those evaluated in **AgentDojo** [12], focus on containing the impact of a compromise through sandboxing but do not address the "Black-box" nature of agentic decisions.

Our framework aligns with the **NIST AI Risk Management Framework (AI RMF 1.0)** [3] and **Zero Trust principles** (SP 800-207) [7], treating every tool call as an untrusted workload. Furthermore, we address the "Store Now, Decrypt Later" (SNDL) threat by adopting NIST-standardized Post-Quantum Cryptography (PQC) primitives [8], [9], providing a temporal dimension of security often overlooked in current agentic research such as **PromptArmor** [4].

III. Threat Model

We define four primary threat vectors for autonomous LLM agents:

1) **Indirect Prompt Injection:** Manipulating the agent's context (e.g., via a malicious website) to execute unauthorized commands [11].
2) **Reasoning Loops (The "Marsh" Problem):** The agent falling into infinite or repetitive tool-calling cycles, leading to resource exhaustion and mission failure.
3) **Identity Spoofing:** An unauthorized process attempting to execute tools by mimicking a legitimate agent session.
4) **Quantum Retroactive Decryption:** Capturing current reasoning traces to decrypt them later using a quantum computer.

IV. Tier 1: Structural Guardrails (Space)

To ensure structural immunity, we adopt a "No-Shell" execution model where system operations are restricted to specialized tools.

A. AST Inspection and Static Analysis

Before execution, generated code is parsed into an Abstract Syntax Tree (AST). A whitelist-based scanner blocks dangerous modules (e.g., `os, socket, pty`) and built-in functions (e.g., `eval, exec`). Specifically, it ensures that `subprocess` calls never use `shell=True`, and detects reflection-based attacks (e.g., `__subclasses__`) to enforce a strict separation between executable logic and data.

B. Environment Isolation via MCP

The framework achieves process isolation by leveraging the Model Context Protocol (MCP) to delegate high-risk tool execution to external environments. By connecting to MCP servers running in dedicated containers, the agent operates in a "Shared-Nothing" architecture, ensuring any malicious activity is contained within a disposable environment.

V. Tier 2: Behavioral & Identity Assurance (Behavior)

A. Workload Identity and ABAC

Each agent instance is assigned a unique key pair. Every MCP tool call is accompanied by a signed **Identity Token** in **COSE** format [5], containing **ABAC** claims. Remote servers verify this token to ensure the request originated from a legitimate agent and enforce strict policies based on workspace attributes.

B. White-box Agency: Cognitive Synchronization

Traditional Human-in-the-Loop (HITL) is often viewed as a bottleneck. We redefine it as **White-box Agency**, where the agent must present its "Internal Monologue" and justification before execution. This serves two purposes:

- **Early Loop Detection:** Humans can detect if an agent is stuck in a reasoning marsh (repetitive trial-and-error) and intervene before resource exhaustion.
- **Context Augmentation:** Users can provide intuitive guidance (e.g., "Check this file instead") at the critical moment of tool invocation.

C. PQC Agility: Context-Adaptive Security Scaling

The framework implements **PQC Agility**, dynamically adjusting security levels based on risk. Higher-risk tools (e.g., `execute_python`) mandate ML-DSA-87 signatures and detailed human approval, while low-risk observations use ML-DSA-44 with relaxed oversight.

D. Bi-directional Verification with ResponseSigner

To ensure the integrity of the entire tool execution loop, we implement **Bi-directional Verification**. While the agent signs the tool request via an Identity Token, high-risk write tools embed a **ResponseSigner** PQC signature (ML-DSA) in their return value, binding the response content to a unique `verification_id`. The `tool_executor` layer verifies this signature before passing the result to the LLM, preventing "Man-in-the-Middle" manipulation of tool outputs and ensuring that the model's subsequent reasoning is grounded in untampered observations.

VI. Tier 3: Post-Quantum Resilience (Time)

A. Quantum-Safe Identity and Hybrid Encryption

We upgrade identity using **ML-DSA** (FIPS 204). Audit logs, containing sensitive reasoning traces, are protected using **ML-KEM-768** (FIPS 203) hybrid encryption.

B. Dual-Layered Audit Architecture

To ensure the long-term integrity of agent history, we implement a two-stage protection model:

- **Stage 1: Hashed Chain (Active Protection):** Each log entry incorporates the hash of the previous entry, preventing real-time revisionism or insertion of malicious traces during an active session.
- **Stage 2: Merkle Tree (Session Anchoring):** Upon session completion or checkpointing, a binary **Merkle Tree**

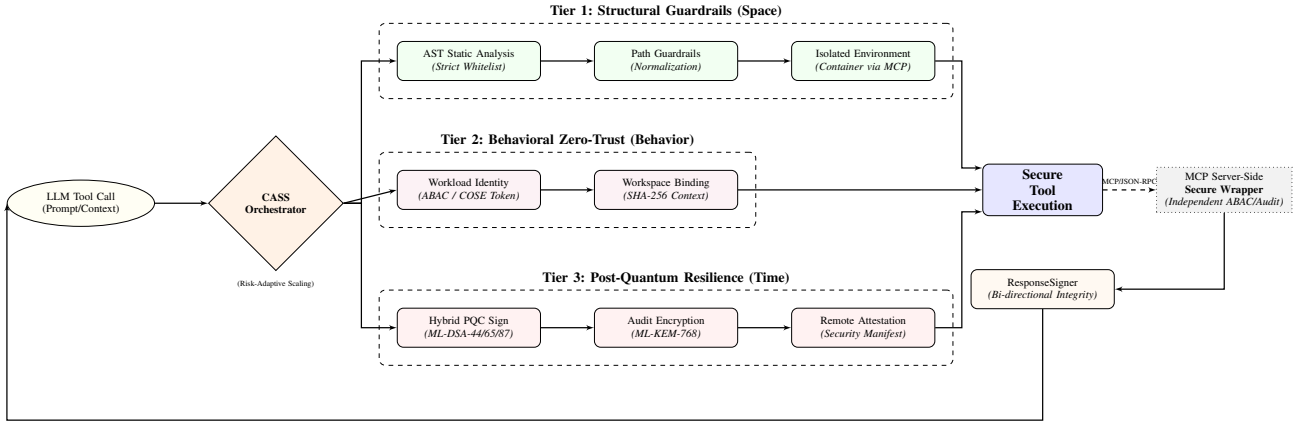

Fig. 2. Detailed System Architecture: Data flow from LLM request through the CASS orchestrator to specific security components across the three tiers, integrating White-box HITL and dual-layered audit.

is computed over all log entries. The resulting Merkle Root serves as a compact, PQC-signed anchor that can be externally verified or published to an immutable log service.

VII. System Architecture and Integration

To synthesize the three tiers into a cohesive architecture, we introduce **Context-Adaptive Security Scaling (CASS)**. CASS serves as the central orchestration engine, dynamically integrating defenses based on the specific risk profile of each tool execution request.

A. CASS Orchestration Logic

CASS determines the required security posture by mapping the requested tool and its arguments to one of three risk levels. As shown in Table I, this mapping ensures that computational resources are allocated proportionally to the security risk.

TABLE I
Tiered Security Enforcement via CASS

Feature	Low Risk	Medium Risk	High Risk
PQC Signature	ML-DSA-44	ML-DSA-65	ML-DSA-87
Audit Protection	Hashed Chain	+ AES-256	+ ML-KEM-768
Human Oversight	Optional	Required	Strict (White-box)
Transparency	Basic	Restrained	Full Justification
AST Analysis	Basic	Restricted	Strict Whitelist

B. Hybrid Identity Tokens (COSE/RFC 9052)

To ensure verifiable workload identity without the overhead of JSON-based JWT, we implement a native **COSE_Sign** structure (CBOR tag 98). Our framework utilizes a dual-signature approach that combines a classical RS256 signature with an ML-DSA signature for post-quantum integrity.

VIII. Evaluation

We evaluated the framework on an AMD Ryzen 5 8540U system (16GB RAM) and a legacy Intel i7-7700 system. The framework's reliability and security protocol compliance were verified through an extensive automated test suite consisting of 191 unit and integration tests, ensuring regression resistance across all three tiers.

A. Security Effectiveness

Table II summarizes the detection capabilities across 20 test vectors (including 4 new distributed trust scenarios). The framework achieved 100% mitigation across all categories in our controlled tests. Notably, the White-box Agency layer enabled users to abort 100% of identified reasoning loops (repetitive trial-and-error cycles) within the first three tool calls, demonstrating the practical value of cognitive synchronization.

TABLE II
Security Effectiveness (n=20 test cases)

Defense Layer	Threat Category	Mitigation Rate
Tier 1 (AST)	Shell/Command Injection	100.0% (6/6)
Tier 1 (AST)	Benign Pass-through	100.0% (3/3)
Tier 2 (White-box)	**Reasoning Loops**	**100.0% (3/3)**
Tier 2 (ABAC)	Unauthorized Workspace	100.0% (3/3)
Tier 2 (Trust)	**Distributed Trust**	**100.0% (4/4)**
Tier 3 (PQC)	Future Decryption	100.0% (Immunity)

B. Performance Latency

As shown in Table III, the cumulative overhead remains negligible compared to typical LLM inference times. Even in the worst-case scenario (high-risk ML-DSA-87 signature + KEM encryption), the total system latency was verified at $\approx$133ms. This "Security Speed Bump" is well within the 500ms–3000ms RTT of cloud-based LLM inference, providing invaluable governance value without degrading the user experience.

TABLE III
SYSTEM LATENCY COMPARISON (MS)

Component	WSL2 (Zen 4)	Native (i7-7700)
AST + Static Analysis	0.03	0.05
Subprocess Overhead	18.50	27.43
Identity Gen (ML-DSA-44)	42.15	68.20
Identity Gen (ML-DSA-87)	112.40	165.30
KEM Encapsulation (L3)	2.86	4.37
Total (Worst-Case)	**133.79**	**197.15**

IX. CONCLUSION

Securing autonomous agents requires a transition from probabilistic "alignment" to a deterministic, human-centric framework. By integrating Structural Guardrails, White-box Agency, and a Dual-Layered PQC Audit architecture, our reference framework demonstrates how AI agency can remain transparent, verifiable, and resilient across spatial and temporal domains. While further independent validation is required for production use, this architecture provides a concrete baseline for high-assurance agentic systems.

Future work will focus on three primary dimensions of hardening. First, we plan to explore the integration of *Trusted Execution Environments* (TEE) such as Intel SGX or AWS Nitro Enclaves to provide "Hardware Sovereignty," isolating the PQC key management and AST analysis from potential host OS compromises. Second, we aim to conduct a formal cryptographic audit of the reference PQC implementations to verify their robustness against side-channel attacks. Finally, we will investigate the integration of these security primitives with corporate governance policies, ensuring that audit retention and access control satisfy diverse legal and regulatory requirements. Through these advancements, the Triple-Lock Framework can serve as a foundational architecture for high-assurance autonomous agent systems.

REFERENCES

[1] Anthropic, "Model Context Protocol (MCP) Specification," 2024.
[2] Protect AI, "LLM-Guard: The Security Toolkit for LLMs," 2023.
[3] NIST, "AI Risk Management Framework (AI RMF 1.0)," 2023.
[4] S. Jain et al., "PromptArmor: Defensive Guardrails for LLM Agents," 2024.
[5] G. Schaad, "CBOR Object Signing and Encryption (COSE): Structures and Algorithms," *RFC 9052*, 2022.
[6] OWASP, "Top 10 for Large Language Model Applications v1.1," 2023.
[7] S. Rose et al., "Zero Trust Architecture," *NIST SP 800-207*, 2020.
[8] NIST, "FIPS 203: ML-KEM Standard," 2024.
[9] NIST, "FIPS 204: ML-DSA Standard," 2024.
[10] F. Perez and I. Ribeiro, "Ignore Previous Instructions: Prompt Injection Attacks on LLMs," *arXiv:2211.09527*, 2022.
[11] K. Greshake et al., "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," *arXiv:2302.12173*, 2023.
[12] E. Debenedetti et al., "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses," *arXiv:2406.13352*, 2024.